

Jetpack Compose第三章第一节教学讲义（状态管理核心）

各位同学，前面我们学过Compose的布局和组件，但如果只停留在静态UI，APP是“活”不起来的——比如点击按钮计数变化、输入框实时显示内容，这些动态效果都依赖“状态”。这节课咱们彻底吃透Compose的状态管理：从“什么是状态”到“如何优雅管理状态”，全程用“生活化比喻+可运行代码”，让你明白状态是如何驱动UI“动”起来的！

一、先搞懂：什么是状态？无状态组件又是什么？

状态是Compose的“灵魂”，先从两个最基础的概念说起，用“饮水机”的例子帮你理解。

1. 什么是状态（State）？

状态就是“UI的动态数据”——它是会变化的，并且变化后会导致UI刷新。就像饮水机的“水量”：

- 水量是“动态数据”（有人接水就减少，加水就增加）；
- 水量变化后，“水位指示灯”会刷新（比如水量低于20%亮红灯）。

在Compose中，“计数APP的数字”“输入框的文本”“开关的选中状态”都是状态。

2. 无状态组件（Stateless Component）

无状态组件就是“不持有状态的组件”——它的UI完全由外部传入的参数决定，就像一个“纯函数”，输入相同，输出必然相同。

Code block

```
1  // 无状态组件：只接收外部参数，不持有状态
2  @Composable
3  fun Greeting(name: String) {
4      // UI完全由name参数决定，参数不变，UI就不变
5      Text(text = "Hello, $name!")
6  }
7
8  // 调用时，传入参数即可
9  @Preview
10 @Composable
11 fun GreetingPreview() {
12     Greeting(name = "小明") // 固定输出"Hello, 小明!"
13 }
```

优势：无状态组件“可预测、易复用、易测试”——因为它没有自己的“小脾气”（状态），完全听外部指挥。

3. 有状态组件（Stateful Component）

有状态组件就是“持有状态的组件”——它内部有自己的动态数据，UI会随内部状态变化而刷新。比如一个“自带计数功能的按钮”：

Code block

```
1  // 有状态组件：内部持有计数状态
2  @Composable
3  fun CounterButton() {
4      // 内部状态：计数（会变化，驱动UI刷新）
5      val count = remember { mutableStateOf(0) }
6
7      Button(onClick = { count.value++ }) {
8          // UI随count变化而刷新
9          Text(text = "点击了 ${count.value} 次")
10     }
11 }
12
13 @Preview
14 @Composable
15 fun CounterButtonPreview() {
16     CounterButton() // 每次点击，文本都会变
17 }
```

注意：有状态组件的UI是“不可预测”的——同样的调用方式，点击后UI会不同，这是它和无状态组件的核心区别。

二、非结构化状态：新手常踩的“状态混乱坑”

很多新手刚开始写有状态组件时，会把所有状态都堆在一个组件里，形成“非结构化状态”——就像把所有东西都堆在桌子上，找的时候一团乱。

1. 非结构化状态的反例

比如一个“登录页面”，把用户名、密码、是否记住密码三个状态都写在页面组件里：

Code block

```
1  // 反例：非结构化状态（所有状态堆在页面组件）
2  @Composable
3  fun LoginPage() {
```

```

4      // 状态1: 用户名
5      val username = remember { mutableStateOf("") }
6      // 状态2: 密码
7      val password = remember { mutableStateOf("") }
8      // 状态3: 是否记住密码
9      val rememberMe = remember { mutableStateOf(false) }
10
11     Column(modifier = Modifier.padding(16.dp)) {
12         OutlinedTextField(
13             value = username.value,
14             onValueChange = { username.value = it },
15             label = { Text("用户名") }
16         )
17         OutlinedTextField(
18             value = password.value,
19             onValueChange = { password.value = it },
20             label = { Text("密码") }
21         )
22         Row {
23             Checkbox(checked = rememberMe.value, onCheckedChange = {
24                 rememberMe.value = it })
25             Text("记住密码")
26         }
27         Button(onClick = { /* 登录逻辑, 需要用到三个状态 */ }) {
28             Text("登录")
29         }
30     }

```

问题：当页面复杂到有10个以上状态时，代码会变得极其混乱，状态之间的依赖关系难以维护，也无法复用状态逻辑。

2. 为什么要避免非结构化状态？

- 维护难：状态分散在代码各处，修改一个状态可能影响多个地方；
- 复用难：状态逻辑和UI绑定，无法抽出来给其他组件用；
- 测试难：无法单独测试状态逻辑，只能连UI一起测。

三、核心思想：单向数据流（Unidirectional Data Flow）

为了解决状态混乱问题，Compose推荐使用“单向数据流”模式——这是一套“状态如何流动”的规则，就像交通规则一样，让状态流动“有序不堵车”。

1. 单向数据流的3个核心规则

1. **状态向下流**：父组件的状态通过参数传递给子组件（子组件只能读，不能改）；
2. **事件向上流**：子组件通过“回调函数”把用户操作（事件）传递给父组件；
3. **状态唯一源**：同一状态只由一个组件持有（避免多个组件修改同一状态导致混乱）。

2. 用“饮水机”理解单向数据流

- 状态向下流：饮水机的“水量”（父状态）显示在“水位灯”（子组件）上；
- 事件向上流：用户“接水”（子组件事件）通过“传感器”（回调）告诉饮水机“水量减少”；
- 状态唯一源：只有饮水机的“水箱”（父组件）持有“水量”状态，水位灯不能直接改水量。

3. 实战：用单向数据流重构登录页面

Code block

```
1  // 1. 子组件：用户名输入框（无状态，只接收状态和回调）
2  @Composable
3  fun UsernameInput(
4      username: String, // 状态向下流：接收父组件的状态
5      onUsernameChange: (String) -> Unit // 事件向上流：回调传递输入事件
6  ) {
7      OutlinedTextField(
8          value = username,
9          onChange = onUsernameChange,
10         label = { Text("用户名") }
11     )
12 }
13
14 // 2. 子组件：密码输入框（无状态）
15 @Composable
16 fun PasswordInput(
17     password: String,
18     onPasswordChange: (String) -> Unit
19 ) {
20     OutlinedTextField(
21         value = password,
22         onChange = onPasswordChange,
23         label = { Text("密码") }
24     )
25 }
26
27 // 3. 父组件：登录页面（持有状态，处理事件）
28 @Composable
29 fun LoginPageWithUdf() {
30     // 状态唯一源：父组件持有所有状态
31     val username = remember { mutableStateOf("") }
```

```

32     val password = remember { mutableStateOf("") }
33     val rememberMe = remember { mutableStateOf(false) }
34
35     Column(modifier = Modifier.padding(16.dp)) {
36         // 状态向下流：传入状态；事件向上流：传入回调
37         UsernameInput(
38             username = username.value,
39             onUsernameChange = { username.value = it }
40         )
41         PasswordInput(
42             password = password.value,
43             onPasswordChange = { password.value = it }
44         )
45         Row {
46             Checkbox(
47                 checked = rememberMe.value,
48                 onCheckedChange = { rememberMe.value = it }
49             )
50             Text("记住密码")
51         }
52         Button(onClick = { /* 登录逻辑 */ }) {
53             Text("登录")
54         }
55     }
56 }

```

 核心改进：子组件变成无状态，只负责“展示UI”和“传递事件”，父组件集中管理状态——状态流动清晰，子组件可复用（比如UsernameInput可在注册页面复用）。

四、Compose状态核心：状态提升与remember、MutableState

前面的例子中用到了 `remember` 和 `MutableState`，还提到了“父组件持有状态”——这就是Compose状态管理的三个核心工具，我们逐一拆解。

1. 状态提升（State Hoisting）：解决状态复用的“利器”

状态提升就是“把状态从子组件移到父组件”，让子组件变成无状态——这是单向数据流的核心实践，就像把“贵重物品”从抽屉（子组件）移到保险柜（父组件），集中管理更安全。

实战：状态提升实现可复用的计数器

```

1 // 1. 子组件：无状态计数器（状态提升到父组件）
2 @Composable
3 fun StatelessCounter(
4     count: Int, // 提升的状态：从父组件接收
5     onIncrement: () -> Unit, // 事件回调：传递点击事件给父组件
6     onDecrement: () -> Unit // 事件回调：传递减少事件
7 ) {
8     Row(modifier = Modifier.padding(16.dp)) {
9         Button(onClick = onDecrement) { Text("-") }
10        Text(text = "当前计数: $count", modifier = Modifier.padding(8.dp))
11        Button(onClick = onIncrement) { Text("+") }
12    }
13 }
14
15 // 2. 父组件：持有状态，控制多个计数器
16 @Composable
17 fun StatefulCounterPage() {
18     // 父组件持有状态1
19     val count1 = remember { mutableStateOf(0) }
20     // 父组件持有状态2
21     val count2 = remember { mutableStateOf(0) }
22
23     Column {
24         // 复用无状态计数器，传入不同状态
25         StatelessCounter(
26             count = count1.value,
27             onIncrement = { count1.value++ },
28             onDecrement = { if (count1.value > 0) count1.value-- }
29         )
30         StatelessCounter(
31             count = count2.value,
32             onIncrement = { count2.value += 2 }, // 不同的逻辑
33             onDecrement = { if (count2.value > 0) count2.value -= 2 }
34         )
35     }
36 }

```

优势：StatelessCounter可无限复用，父组件可给不同的计数器设置不同的逻辑（比如一个加1，一个加2），状态管理极其清晰。

2. remember：让状态“记住”值（避免重组丢失）

很多新手会疑惑：“为什么状态要用remember包装？直接用var不行吗？”——答案是：Compose会频繁“重组”，不用remember的话，状态会被重置。

反例：不用remember，状态会丢失

Code block

```
1  @Composable
2  fun CounterWithoutRemember() {
3      // 错误：不用remember，重组后count会重置为0
4      var count = 0
5
6      Button(onClick = { count++ }) {
7          Text("点击了 $count 次")
8      }
9  }
```

原因：Compose的重组会重新执行Composable函数——每次点击按钮，函数都会重新执行，count会被重新赋值为0，所以状态无法保留。

正例：用remember保存状态

Code block

```
1  @Composable
2  fun CounterWithRemember() {
3      // 正确：用remember保存状态，重组后不会重置
4      val count = remember { mutableStateOf(0) }
5
6      Button(onClick = { count.value++ }) {
7          Text("点击了 ${count.value} 次")
8      }
9  }
```

原理：remember会把状态“存储”在Compose的“重组作用域”中，即使函数重新执行，也会拿到之前存储的状态值。

3. MutableState：让状态“可观察”（触发UI刷新）

remember解决了“状态保留”问题，但要让“状态变化触发UI刷新”，还需要 `MutableState`——它是Compose的“可观察状态容器”。

核心原理：MutableState的“观察者模式”

- 当你用 `mutableStateOf` 创建状态时，Compose会自动“观察”这个状态；
- 当状态的 `value` 变化时，Compose会找到使用该状态的UI部分，触发局部重组（只刷新用到状态的地方）。

简化写法：by委托（更优雅的状态访问）

每次写 `count.value` 比较繁琐，可通过 `by` 委托简化，需要先导入依赖：

Code block

```
1 // 导入by委托的依赖
2 import androidx.compose.runtime.getValue
3 import androidx.compose.runtime.setValue
4
5 @Composable
6 fun CounterWithBy() {
7     // by委托：直接用count代替count.value
8     var count by remember { mutableStateOf(0) }
9
10    Button(onClick = { count++ }) { // 不用写count.value++
11        Text("点击了 $count 次") // 不用写count.value
12    }
13 }
```

注意：`by` 只是语法糖，本质和 `mutableStateOf` 一样，目的是让代码更简洁。

五、重组：状态驱动UI的“幕后推手”

前面反复提到“重组”，现在我们彻底讲透它——重组是Compose“状态变化触发UI刷新”的核心机制，但它不是“全量刷新”，而是“精准刷新”。

1. 重组的本质：“只刷新需要变化的部分”

就像你写作文时，发现一个错别字，只需要修改那个字，不用重写整篇文章——Compose的重组也是如此。

Code block

```
1 @Composable
2 fun RecompositionExample() {
3     var count by remember { mutableStateOf(0) }
4     val name = "小明" // 固定值，不参与重组
5
6     Column {
7         // 用到count, count变化时会重组
8         Text(text = "计数: $count")
9         Button(onClick = { count++ }) {
10             Text("增加")
11         }
12         // 没用到count, count变化时不会重组
13         Text(text = "用户名: $name")
14     }
```


逻辑解析：点击按钮时，只有显示“计数”的Text和Button会重组，显示“用户名”的Text完全不动——这种“局部重组”保证了Compose的高性能。

2. 避免“不必要的重组”

虽然Compose的重组性能很高，但我们还是要避免不必要的重组，核心原则是：**让Composable函数的参数尽量稳定。**

Code block

```
1  // 优化前：每次重组都会创建新的Lambda，导致子组件重组
2  @Composable
3  fun BadExample() {
4      var count by remember { mutableStateOf(0) }
5
6      // 问题：每次重组都会新建一个{ count++ }，导致Child重组
7      Child(onClick = { count++ })
8  }
9
10 // 优化后：用remember保存Lambda，避免新建
11 @Composable
12 fun GoodExample() {
13     var count by remember { mutableStateOf(0) }
14     // 用remember保存回调，避免每次重组新建
15     val increment = remember { { count++ } }
16
17     Child(onClick = increment)
18 }
19
20 // 子组件
21 @Composable
22 fun Child(onClick: () -> Unit) {
23     Button(onClick = onClick) {
24         Text("增加")
25     }
26 }
```

六、小节回顾：状态管理核心知识点

1. 核心概念速查表

--	--	--

概念	核心作用	关键用法
无状态组件	接收外部参数，无内部状态，可复用	UI完全由参数决定，不使用mutableStateOf
有状态组件	持有内部状态，驱动UI刷新	用remember+mutableStateOf管理状态
单向数据流	规范状态流动，避免混乱	状态向下流（参数），事件向上流（回调）
状态提升	将子组件状态移到父组件，实现复用	子组件接收状态和回调，父组件持有状态
remember	保存状态，避免重组丢失	remember{ mutableStateOf(初始值) }
MutableState	可观察状态，触发局部重组	用by委托简化为var count by ...

2. 状态管理 “黄金法则”

- 1. 优先用无状态组件：能通过参数传递的，就不要在组件内部定义状态；
- 2. 状态尽量提升：把状态提升到 “需要使用该状态的最顶层父组件” ；
- 3. 遵循单向数据流：子组件不直接改状态，只通过回调传递事件；
- 4. 用remember优化性能：保存状态、回调等，避免不必要的重组。

七、课后作业：动手实现 “待办事项（Todo） 列表”

需求：

- 1. 定义数据模型 `TodoItem` ，包含属性：id (Int)、内容 (content: String)、是否完成 (isCompleted: Boolean) ；
- 2. 实现三个组件：

`TodoInput`（无状态）：输入待办内容的文本框+添加按钮，接收 “输入内容” 状态和 “添加回调” ；
- 3. `TodoItem`（无状态）：单个待办项，包含复选框（控制isCompleted）和内容文本，接收 `TodoItem`对象和 “状态变化回调” ；
- 4. `TodoListPage`（有状态）：持有待办列表状态，组合`TodoInput`和`TodoItem`，处理添加、修改待办的逻辑；
- 5. 实现功能：添加待办、点击复选框标记完成/未完成、删除已完成的待办；

6. 用by委托简化状态访问，用remember优化回调，避免不必要的重组。

提示：待办列表状态可用 `mutableStateListOf<TodoItem>()`（可观察的列表），添加待办时调用 `add()`，删除时调用 `removeAll()`。重点练习状态提升和单向数据流的实践，确保子组件都是无状态的。

（注：文档部分内容可能由 AI 生成）